An Exploratory Taxonomy of Use Cases for Zero-Liquidity Micro Payments Utilizing Cryptocurrency Web Mining*

Fajar Purnama[1], Adi Juliawan Pawana[1], and Adi Panca Saputra Iskandar[1]

Udayana University, Bali, Indonesia
{fajarpurnama,adijuliawanpawana,adipanca}@unud.ac.id

Abstract

The structural architecture of modern e-business monetization is fundamentally constrained by the fiat "Penny Gap" and the Mental Transaction Costs (MTC) associated with managing Web3 digital wallets. This manuscript explores "zero-liquidity" computational payments, utilizing browser-based cryptocurrency web mining to replace financial capital with computational attestation. Employing an exploratory landscape mapping methodology that synthesizes multivocal literature—including formal academic research, industry documentation, and non-formal developer innovations—this study identifies a comprehensive array of historical and contemporary web mining applications. These applications are formalized into an exploratory taxonomy comprising four foundational pillars: Content Monetization, Philanthropy, Programmatic Utilities, and Digital Security. To transition these theoretical concepts into actionable e-business integrations, this manuscript designs a modular front-end architecture. Utilizing Object-Oriented JavaScript and Unified Modeling Language (UML) diagrams, we propose a Minimum Viable Product (MVP) framework that integrates with a backend Stratum proxy. Finally, we subject the MVP to an empirical hardware evaluation on a live Mintme blockchain network, capturing kernel-level system telemetry to profile CPU saturation, power consumption, and network footprints. We combine these metrics with real-world Mintme blockchain difficulty parameters to mathematically model the economic feasibility of zero-liquidity transactions across four e-business scenarios. The results outline infeasibility for most end user hardware but remained feasible for high specifications ones and still shows potential pathways for future deployment, cybersecurity mitigation, and standardized Proof-of-Consent protocols.

1 Introduction

Current e-business monetization heavily relies on two primary models: intrusive advertising and recurring subscriptions. Both models create significant friction for end users. Advertising often degrades the user experience and raises privacy concerns, while the increasing number of paywalls has led to widespread subscription fatigue. Consumers are often unwilling to commit to monthly fees just to access disparate digital assets or single pieces of content.

A logical alternative to these models is the micropayment. A micropayment is a financial framework that allows users to pay fractions of a dollar for individual actions, such as reading an article, downloading a file, or submitting a form. However, traditional fiat payment gateways, such as credit cards and bank transfers, impose base transaction fees. These fixed fees instantly negate the profit margin of micro-transactions, creating a phenomenon known as the "Penny Gap," [1] where transactions under $0.50 become economically unviable for the platform to process.

*Proceedings of The 2026 IFIP WG 8.4 International Symposium on E-Business Information Systems Evolution (EBISION 2026), Article No. 42, July 8-10, 2026, Seoul, South Korea.
© The copyright of this paper remains with the author(s).

To solve this fiat micropayment problem, various decentralized digital asset frameworks and cryptocurrencies have been proposed because they offer extremely low network fees. Yet, these solutions introduce a new, severe barrier for the general public: the liquidity requirement. They require the end user to possess a funded digital wallet before interacting with the system. Forcing a standard user to purchase, transfer, and manage cryptocurrency just to pay $0.05 for an article introduces steep psychological friction, often referred to as Mental Transaction Costs (MTC) [2]. Because the mental effort outweighs the value of the digital asset, users typically abandon the transaction.

An alternative approach to overcome both the Penny Gap and the liquidity requirement is the "zero-liquidity" computational payment. Recent research has established the technical feasibility of utilizing consent-based JavaScript and WebAssembly (WASM) browser mining as a direct method of payment. By utilizing CPU-friendly Proof-of-Work algorithms, users can compensate e-business platforms in real-time using their device's computational cycles rather than spending stored financial assets.

While the backend technical feasibility of WASM web mining has been proven in our previous work, the integration of this technology into practical e-business front-ends remains underexplored. Therefore, this manuscript explores use cases for zero-liquidity micro payments utilizing cryptocurrency web mining as a method of payment.

To bridge the gap between technical feasibility and real-world application, this paper makes the following contributions:

1. It conducts an exploratory landscape mapping using a multivocal approach to identify a comprehensive array of web mining use cases across different e-business domains.

2. It synthesizes these use cases into an exploratory taxonomy based on the specific friction they resolve, establishing four operational pillars.

3. It designs the system architecture for these use cases through Unified Modeling Language (UML) diagrams and builds their front-end Minimum Viable Products (MVPs).

4. It executes an empirical evaluation of the front-end architecture, profiling system-level resource consumption (CPU, network, and thermals) and providing macroeconomic feasibility models calibrated against live blockchain network difficulty constraints.

The remainder of this manuscript is organized as follows: Section 2 reviews the foundational previous work regarding browser-based web mining. Section 3 details the multivocal methodology used for landscape mapping and empirical testing. Section 4 presents the taxonomy of use cases alongside their UML designs and MVP front-ends. Finally, Section 6 critically evaluates the system's economic feasibility, performance benchmarking, real-world hardware limitations, and future outlook.

2 Previous Work: Foundations of Browser-Based Web Mining

Our previous work [3] highlights that cryptocurrency web mining can be used for online payments. The general method operates as follows: the merchant sets a price, the customer mines cryptocurrency on behalf of the merchant until the price is reached, the mined cryptocurrency goes directly to the merchant, and finally, the customer receives the item. To demonstrate this, we built a minimum viable product (MVP) each in standard HyperText Markup Language

(HTML) and JavaScript utilizing CoinIMP's platform, Marco Ciaramella's cpu-webminer, and additionally WebAssembly (WASM) from MintMe's open-source miner still available in Github repository [1].

Ideally, the merchant sets the price in a specific cryptocurrency amount, or even more difficult, converts it to a stable currency like stablecoins. However, our previous work has practically not reached that possibility. While solo mining pays a higher reward, finding a block is difficult and randomly rare. Joined mining in a mining pool offers smaller but more frequent rewards. Because of this dynamic, our base MVP currently requires the merchant's price to be set in either *hashes* or *shares*. Hashes represent the raw amount of computational work a customer does, while shares represent randomly found valid work submitted to the pool. Using hashes as the price is advantageous for the customers because the cost is clear and deterministic for them, but it is disadvantageous for the merchants because they are not guaranteed to receive the final cryptocurrency payout (since raw hashes do not guarantee valid shares). Conversely, using shares as the payment price protects the merchant but creates an unpredictable time cost for the customer.

While our previous work successfully proved the technical backend feasibility of connecting a web browser to a mining pool, it lacked exploration into diverse front-end user experiences. The previous MVPs were primarily technical proofs-of-concept. The study did not map out specific e-business scenarios where this technology is most beneficial. Therefore, building upon that technical foundation, this current manuscript focuses entirely on exploring the practical e-business use cases and designing their specific front-end architectures.

3 Methodology

This study does not conduct a standard Systematic Literature Review (SLR). Relying solely on an SLR would omit critical empirical data from industry deployments, developer post-mortems, and open-source projects because the technological landscape of cryptocurrency web mining evolves faster than formal academic publication cycles.

Instead, this manuscript conducts an exploratory landscape mapping methodology utilizing a multivocal approach. A multivocal approach allows the synthesis of data from diverse, non-traditional sources. In this study, we collect and analyze formal academic research alongside "gray literature," which includes industry documentation, whitepapers, GitHub repositories, and non-formal innovations. This method is necessary to identify a comprehensive array of real-world and theoretical cryptocurrency web mining use cases.

The methodology is executed through three main phases as detailed below.

3.1 Phase 1: Exploratory Landscape Mapping

The first phase involves synthesizing the multivocal literature to discover and list potential zero-liquidity web mining use cases. To do this, we manually searched open-source repositories (such as GitHub), developer discussions, and historical archives of past web mining platforms. We filtered these sources to strictly include use cases where the end user provides computational power rather than financial liquidity. Finally, we map these gathered use cases across different e-business domains to understand where computational payments can effectively replace traditional fiat micro-transactions.

To aggregate this multivocal literature, this study utilized an AI-assisted search mechanism (Google Gemini). The search was strictly bounded by specific inclusion and exclusion criteria:

[1] https://github.com/dappsourcecode/Cryptocurrency-Miners

it explicitly excluded traditional decentralized wallets (e.g., Lightning Network) and exclusively targeted implementations where the end user provides 'zero-liquidity' computational power via browser mining.

3.2 Phase 2: System Architecture Design

The second phase involves grouping the discovered use cases into an exploratory taxonomy based on the specific friction they resolve. After grouping them, we model the technical flow for each taxonomic category using Unified Modeling Language (UML). Specifically, we utilize UML Sequence Diagrams to illustrate the chronological sequence of events. These diagrams map exactly how the user interacts with the user interface, how the browser communicates with the underlying web mining proxy, and how the merchant's backend verifies the payment.

3.3 Phase 3: MVP Development

The final phase focuses on practical implementation. We build front-end minimum viable products (MVPs) in standard HTML and vanilla JavaScript. These MVPs integrate the WebAssembly (WASM) mining modules established in our previous work. The prototypes are tested locally to ensure they successfully connect to a mining pool, calculate the required hashes or shares, and trigger the intended front-end action (such as unlocking content or authorizing a form submission). These MVPs serve to demonstrate the practical integration of the modeled use cases, proving that the theoretical taxonomy can be successfully applied to real-world e-business front-ends.

3.4 Phase 4: Empirical Evaluation and Economic Modeling

The final phase transitions from sandbox functional validation to real-world performance profiling and macroeconomic benchmarking. To address the physical realities of deploying WebAssembly mining on consumer devices, this phase is executed in two parallel vectors:

- **System-Level Telemetry Profiling:** The vanilla JavaScript MVP is deployed onto a consumer hardware testbed to measure real-world performance under strict device constraints. Using Linux system profiling utilities (`vmstat`, `turbostat`, and `nethogs`), we continuously monitor kernel-level metrics, including user-space CPU saturation, package power consumption, processor thermal states, and network bandwidth overhead. This empirical data quantifies the physical "cost" of zero-liquidity payments transferred to the user's hardware.

- **Macroeconomic Scalability Modeling:** Utilizing live blockchain parameters extracted directly from the target network's explorer and mining pool (including network difficulty, active pool difficulty, block rewards, and market exchange rates), we mathematically define the exact fiat-equivalent value of a single computational share. This empirical baseline is then used to model the transaction time-to-unlock across distinct e-business scenarios, comparing legacy mobile architectures against modern high-throughput desktop processors to evaluate commercial viability.

4 An Exploratory Taxonomy of Use Cases

This section groups the discovered web mining use cases into an exploratory taxonomy based on the specific e-business friction they resolve. For each subsection, we explore the historical and current implementations, comparing traditional methods that require financial liquidity against zero-liquidity alternatives.

4.1 Content Monetization

Content monetization is the most direct application for replacing fiat micro-subscriptions and intrusive advertising.

- **Content Creator Tipping:**
  - *Requires Liquidity:* Traditional tipping requires mobile banking transfers, PayPal, or standard cryptocurrency transfers, all of which impose a "Penny Gap" fee.
  - *Zero Liquidity:* Utilizing our previous work's cryptocurrency miner running on a website. Historically, platforms like JSEcoin attempted to formalize this ecosystem, allowing publishers to receive tipping rewards via background processing, effectively monetizing traffic without requiring the user's credit card [4].
- **File Access, Download, and Selling:**
  - *Requires Liquidity:* Mobile banking or PayPal are technically possible but the feature must be built by providers. Currently, true micropayments are non-existent in traditional electronic finance systems. For cryptocurrency, this is possible through decentralized finance (DeFi) such as smart contracts or token holding subscriptions. Other than tokens, NFTs can be utilized to serve as a pass.
  - *Zero Liquidity:* Utilizing our previous work's file access scenario where a download is granted only when a specific mining requirement (e.g., a set amount of valid shares) is met.
- **Premium Feature Gating:**
  - *Requires Liquidity:* Requires standard subscription methods, such as a credit card charged annually. For cryptocurrency can utilize decentralized finance (DeFi) such as smart contracts or token holding mechanism where the more token hold the more contents unlocked. Other than tokens, NFTs can be utilized to serve as a pass.
  - *Zero Liquidity:* An ethical "Mine to Unlock" model where users explicitly consent to exchange computation for access. Use cases include mining to remove advertisements, mining per view, and having premium text content gradually unblur the more the user mines. The webminer project by opd-ai exemplifies this by providing a framework that tracks user progress and prevents device overheating while gating features [5].
- **E-Commerce Discounts and Loyalty Rewards:**
  - *Requires Liquidity:* Users must pay for a premium membership or buy a specific volume of goods to unlock discounts and loyalty points.

  - *Zero Liquidity:* Exchanging time and computation for retail savings. A user can leave a tab open to mine on the merchant's site in exchange for generating a percentage discount code at checkout.

- **Interactive Media and Gaming:**
  - *Requires Liquidity:* Micro-transactions in gaming require players to use credit cards to purchase in-game lives, cosmetic skins, or loot boxes.
  - *Zero Liquidity:* A "Mine-to-Play" or "Mine-to-Revive" model. Players can opt to mine for a predetermined duration to earn an extra life or unlock cosmetic items without ever linking financial details.

4.2 Philanthropy

Philanthropy is a compelling use case for zero-liquidity payments because it allows individuals without financial means to contribute to charitable causes.

- **Donation and Global Humanitarian Aid:**
  - *Requires Liquidity:* Providing a financial account address either in the form of text or a Quick Response (QR) code in social media and other media. This is heavily impacted by fiat processing fees which consume sub-dollar donations. For cryptocurrency are affected only by network fees and after that for example exchanging cryptocurrency the fees depends.
  - *Zero Liquidity:* Providing a Uniform Resource Locator (URL) to a web miner in social media. Real-world implementations have successfully proven this model. For example, Bail Bloc allowed users to donate spare CPU cycles to mine Monero, which was converted to USD to fund bail for individuals in the criminal justice system [6,7]. Similarly, UNICEF Australia's "Hopepage" utilized a web dashboard where users adjusted a CPU intensity slider to mine and donate directly to humanitarian aid, bypassing traditional donation minimums [8].

4.3 Utilities (Programmatic Services)

This category involves machine-to-machine payments and programmatic access to digital resources.

- **Examples:** Payments for Artificial Intelligence (AI) tasks or prompts, Application Programming Interface (API) requests, and Internet of Things (IoT) live sensing data.
- *Requires Liquidity:* For traditional electronic finance, this relies solely on annual or monthly subscription models. Cryptocurrencies additionally have a pay-as-you-go model through DeFi smart contracts, but still require a funded wallet. Other than tokens, NFTs can be utilized to serve as a pass.
- *Zero Liquidity:* A "mine-as-you-go" model. For AI and API requests, protocols like x402 leverage the HTTP 402 "Payment Required" status code, allowing autonomous AI agents to pay for API access by solving Proof-of-Work challenges [9]. For IoT, the "Sensing-as-a-Service" (S2aaS) model allows sensors to act as nodes that are compensated for their environmental data through background computational verification [10].

4.4 Digital Security

Digital security usually does not require financial liquidity. However, cryptocurrency web mining can replace or be better integrated into current digital security systems by treating computation as a form of attestation rather than currency.

- **Spam Deterrence and Access Control:**
  - *Requires Liquidity:* Traditional security relies on Completely Automated Public Turing tests to tell Computers and Humans Apart (CAPTCHA), which creates high user friction, or centralized subscription services for API rate-limiting.
  - *Zero Liquidity:* Utilizing Proof-of-Work (PoW) as computational attestation. Implementations like Altcha, mCaptcha, and Friendly Captcha force the user's browser to solve a cryptographic puzzle in the background, ensuring every request carries a real computational cost [11–13]. Furthermore, specialized firewalls like Anubis serve SHA-256 challenges to incoming requests, making it economically unfeasible for AI crawlers or bots to scrape a website while remaining transparent to legitimate human users [14].

Table 1: Summary of the Exploratory Taxonomy for Zero-Liquidity Web Mining

Taxonomy Category	**Example Use Cases**	**Primary E-Business Friction Resolved**
Content Monetization	Tipping, File Access, Premium Gating, Gaming Micro-transactions	Eliminates the fiat "Penny Gap" fee; removes the psychological barrier of entering credit card details.
Philanthropy	Global Humanitarian Aid, Micro-Donations	Enables sub-dollar contributions and allows users without disposable income to donate computational value.
Utilities (Programmatic)	API Access, AI Prompt Execution, IoT Sensing	Allows pay-as-you-go machine-to-machine transactions without requiring pre-funded Web3 wallets.
Digital Security	Spam Deterrence, Bot Mitigation, PoW CAPTCHAs	Replaces visually intrusive CAPTCHAs with transparent, economically unfeasible computational costs for attackers.

5 System Architecture and MVP Implementation

To translate the exploratory taxonomy into practical e-business applications, it is necessary to design a flexible system architecture. Building upon the technical backend established in our previous work, this section details the network sequence and the modular front-end Minimum Viable Product (MVP) built using standard HTML and vanilla JavaScript.

5.1 Conceptual Network Architecture

Regardless of the specific use case (e.g., file access, content tipping, or API requests), the fundamental network interaction remains consistent. The architecture consists of four primary

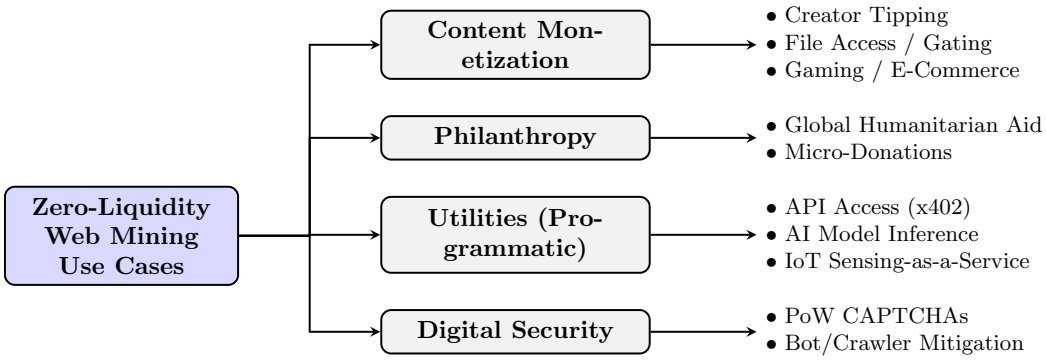


Figure 1: Hierarchical taxonomy of zero-liquidity web mining use cases.

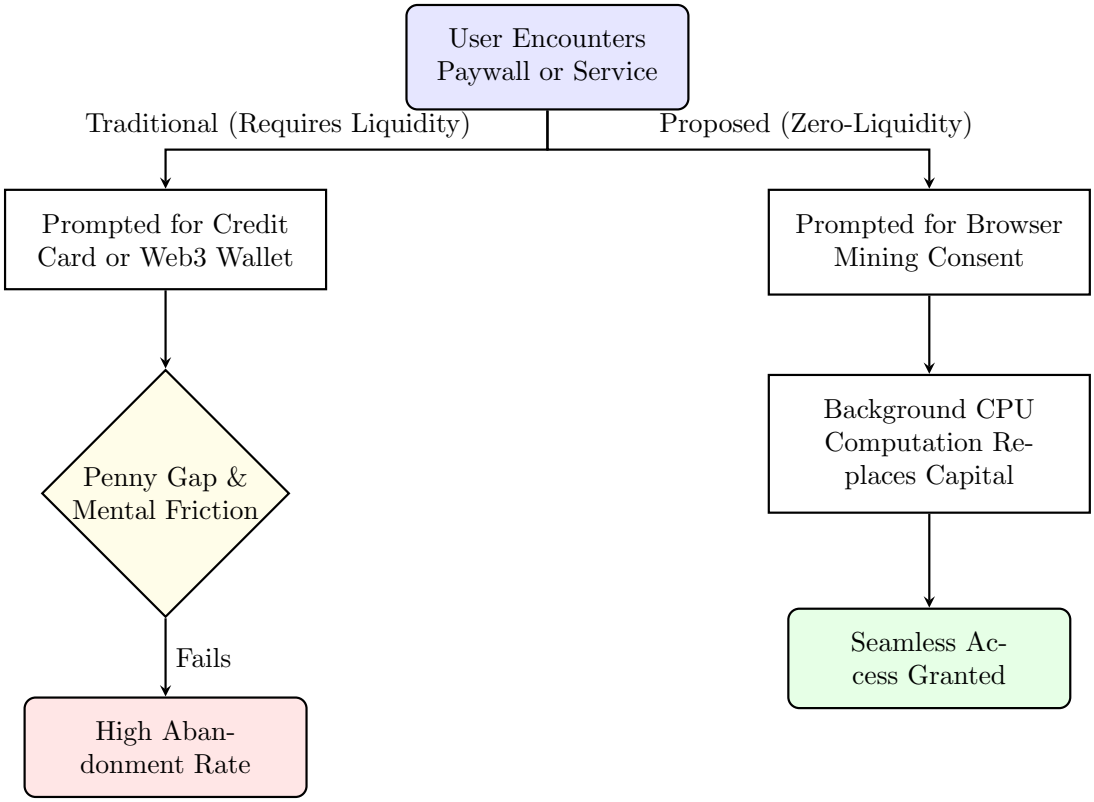


Figure 2: User journey comparison highlighting the mitigation of mental transaction costs.

actors: the Client (user's browser running the JS MVP), the Merchant Server, the Stratum Proxy (the Python bridge from our previous work), and the external Mining Pool.

The interaction follows a structured sequence. Initially, the client requests a gated resource. The merchant server denies access and returns a specific "target price" that must be met. The client then establishes a WebSocket connection to the Stratum Proxy, which translates the connection into standard TCP Stratum messages for the Mining Pool. As the user's browser calculates cryptographic hashes and finds valid shares, it submits them back through the proxy. Once the accumulated computational work reaches the target price, the merchant server releases the resource. This flow is illustrated in the sequence diagram in Figure 3.

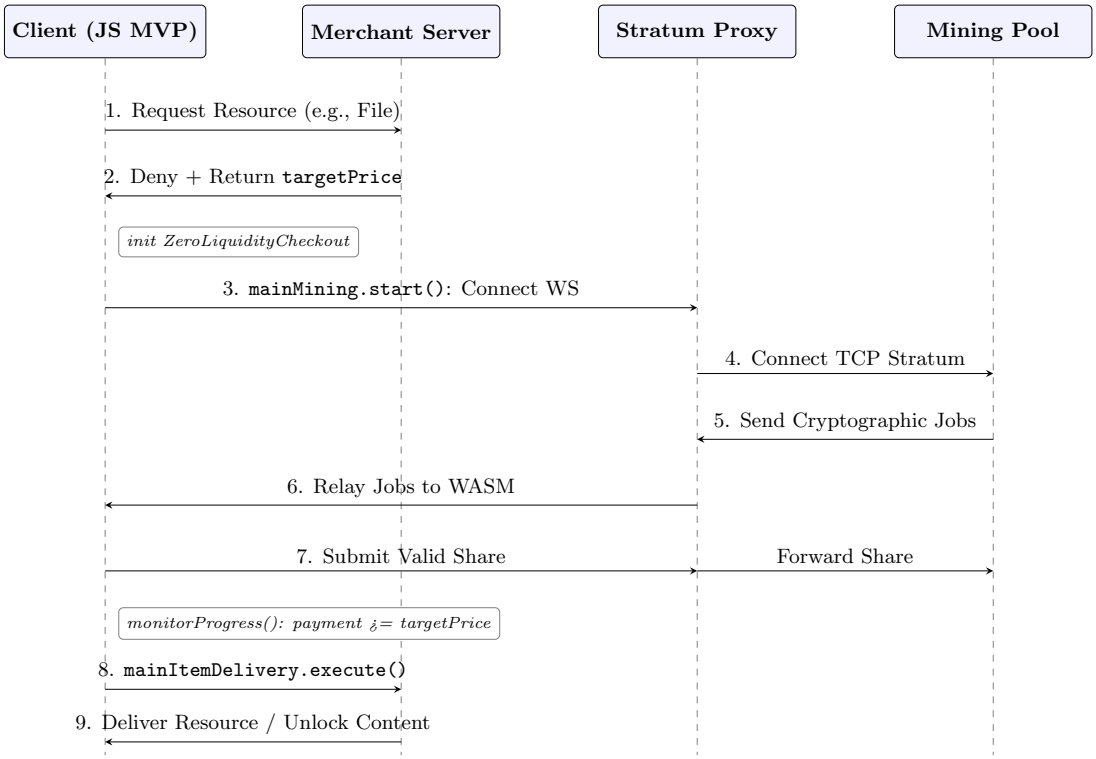


Figure 3: UML Sequence Diagram integrating the JavaScript modular architecture with the backend web mining network.

5.2 Front-End Modular Design

A critical challenge in developing this MVP is ensuring that the codebase is adaptable across the different use cases identified in our taxonomy. To solve this, we implemented a modular "Separation of Concerns" principle using Object-Oriented JavaScript.

The front-end architecture is governed by a main controller class (e.g., `ZeroLiquidityCheckout`) that monitors three core variables:

1. `targetPrice`: An integer set by the merchant.
2. `pricingMetric`: A string set to either "hashes" or "shares" to determine what computational metric is being tracked.

3. `payment`: A real-time integer tracking the user's progress.

To ensure flexibility, the core mining execution and the final item delivery are decoupled from the main controller using interface adapters. The `mainMining` adapter contains the specific logic to connect to different mining networks (e.g., MintMe WASM or CoinImp). The `mainItemDelivery` adapter contains the specific logic for the user's reward (e.g., unlocking a file or initiating an AI prompt).

By utilizing this structure, a merchant can easily swap the underlying mining backend or change the delivered item without breaking the payment monitoring logic. This architecture is visualized in the class diagram in Figure 4.

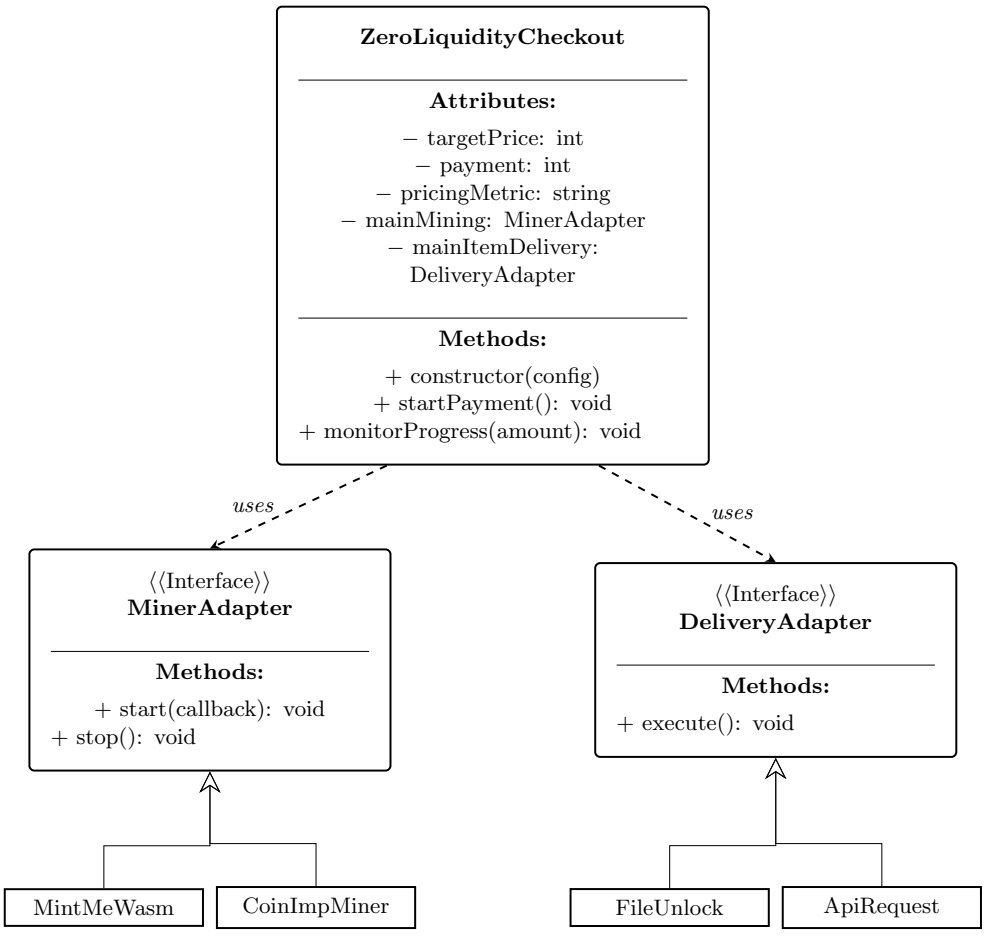


Figure 4: UML Class Diagram illustrating the modular Vanilla JavaScript architecture for the front-end MVP. The main controller delegates logic to swappable Miner and Delivery adapters.

5.3 JavaScript Implementation Logic

To demonstrate the practical application of the class architecture, we developed a vanilla JavaScript MVP. The core logic relies on the main controller initializing the target variables and injecting the chosen adapters.

The code snippet in Listing 1 demonstrates the foundational logic of the checkout controller. The `startPayment()` method triggers the mining adapter, which continuously returns the calculated hashes or shares. The `monitorProgress()` method adds these to the `payment` variable until it meets or exceeds the `targetPrice`, at which point the mining loop is terminated and the delivery adapter is executed.

```
class ZeroLiquidityCheckout {
    constructor(config) {
        this.targetPrice = config.targetPrice;
        this.payment = 0;
        this.pricingMetric = config.pricingMetric;

        this.mainMining = config.mainMining;
        this.mainItemDelivery = config.mainItemDelivery;
    }

    startPayment() {
        this.mainMining.start((minedAmount) => {
            this.monitorProgress(minedAmount);
        });
    }

    monitorProgress(minedAmount) {
        this.payment += minedAmount;

        if (this.payment >= this.targetPrice) {
            this.mainMining.stop();
            this.mainItemDelivery.execute();
        }
    }
}

// Example Merchant Deployment:
const myCheckout = new ZeroLiquidityCheckout({
    targetPrice: 50,
    pricingMetric: "shares",
    mainMining: new MintMeWasmMiner('proxy.merchant.com'),
    mainItemDelivery: new FileUnlockDelivery('premium_file.pdf')
});
```

Listing 1: Vanilla JavaScript implementation of the modular MVP controller.

This modular MVP proves that zero-liquidity computational payments can be smoothly integrated into existing e-business front-ends with minimal merchant friction. To encourage further academic review and commercial adoption, the complete, open-source JavaScript implementation of this architecture is available via the project's GitHub repository[2].

5.4 Mathematical Valuation of Computational Shares

To dynamically translate fiat-based e-business prices into zero-liquidity computational requirements, the system must calculate the real-time monetary value of a single submitted hash share. This valuation relies on network parameters, pool difficulty, and current market exchange rates. The calculation follows a three-step formulation.

First, the expected cryptocurrency yield of a single computational share, denoted as E_{coin}, is calculated by evaluating the probability of the share solving a block, scaled by the block

[2] https://github.com/dappsourcecode/Cryptocurrency-Miners/webapp/mintme/module

reward and deducting the proxy pool fee:

$$E_{coin} = R \times \left(\frac{D_p}{D_n} \right) \times (1 - f)$$

Where:

- R is the current block reward of the blockchain network.
- D_p is the dynamically assigned pool difficulty (the target hash threshold required for the proxy to accept a share).
- D_n is the overall network difficulty.
- f is the percentage fee deducted by the mining pool or Stratum proxy.

Second, this expected cryptocurrency yield is converted into its fiat equivalent (e.g., USD), denoted as E_{usd}, utilizing a real-time price oracle or exchange API:

$$E_{usd} = E_{coin} \times P_{coin}$$

Where P_{coin} represents the current market price of one unit of the mined cryptocurrency in USD.

Finally, to bridge the "Penny Gap," the merchant's target fiat price for the digital good or service (T_{fiat}) is divided by the expected fiat value of a single share. This determines the exact target number of shares (S_{target}) the user's browser must compute to unlock the asset:

$$S_{target} = \left\lceil \frac{T_{fiat}}{E_{usd}} \right\rceil$$

Estimation Example: Utilizing real-world parameters from the MintMe network [15, 16], assume a current block reward (R) of 0.720029 MINTME, an overall network difficulty (D_n) of 1.37 MH (1,370,000 hashes), a pool-assigned difficulty (D_p) of 12,500 (standard for Stratum port 2222), and a standard pool fee (f) of 1% (0.01). Under these conditions, a single valid share yields an expected value of approximately 0.0065 coins. Given a market exchange rate (P_{coin}) of \$0.0005 [17], a single share possesses a fiat equivalent of \$0.00000325 (E_{usd}). To unlock a \$0.05 digital file, the application controller dynamically calculates S_{target}, requiring the user's web miner to submit approximately 15,384 valid shares at this specific difficulty tier.

6 Discussion and Limitations

The exploratory taxonomy and the subsequent MVP architecture demonstrate that zero-liquidity computational payments are technically feasible. However, transitioning from fiat transactions to computational attestation introduces novel economic paradigms, ethical considerations, and strict hardware limitations that must be addressed prior to commercial deployment.

6.1 The Shift in E-Business Value Exchange

The primary implication of this research is the conceptual shift in how e-business value is extracted. Traditional monetization models require the user to possess and transfer capital (fiat

or cryptocurrency), inherently excluding demographics without access to banking infrastructure. By replacing capital with computation, e-businesses tap into the "latent value" of a user's device. This democratizes access to premium content and bypasses both the fiat "Penny Gap" and the Mental Transaction Costs (MTC) associated with manual Web3 wallet authorizations.

6.2 Ethical Considerations and Proof-of-Consent

A critical barrier to commercial adoption is the "Coinhive Stigma." Historically, the surreptitious deployment of web miners (cryptojacking) led to a severe backlash from the cybersecurity community. Consequently, modern antivirus software and browser ad-blockers routinely flag WebAssembly mining scripts by default. To operate legitimately, merchants must implement rigorous "Proof-of-Consent" mechanisms. As demonstrated in our MVP UI shown in Figure 5, mining must never run hidden in the background; it requires an explicit user action (e.g., clicking an "Attest Computation" button) accompanied by transparent real-time metrics showing the user exactly what their device is doing.

6.3 Empirical Hardware Evaluation and System Constraints

To observe real-world probabilistic payout variance and system constraints, an empirical bench test of the JavaScript MVP was conducted on an older mid-tier laptop (Intel™ Core i7-4500U, 12.0 GiB RAM, Ubuntu 24.04.4 LTS). The web miner was restricted to 3 Web Workers. The objective was to compute and submit 10 accepted shares to the MintMe network. The results are summarized in Table 2.

Table 2: Empirical Benchmarking Results for 10 MintMe Shares

Parameter	**Observed Value**
Hardware	ASUSTeK X450LCP (Intel i7-4500U, 12GB RAM)
Mining Configuration	3 Web Workers (WASM Lyra2v2-webchain)
Pool-Side Hashrate (30m Avg)	69.00 H/s
Total Time to 10 Shares	28 minutes, 4 seconds
Average Time per Share	$\approx$ 168.4 seconds (2.8 minutes)
Shortest Share Interval	12 seconds
Longest Share Interval	8 minutes, 53 seconds

6.3.1 Probabilistic Payment Timing

Resource monitoring screenshot at start of mining is provided on Figure 6 for readability easiness of this manuscript but the raw monitored log data of whole process is available in Github repository directory[3]. It heavily reinforces the limitation of probabilistic payment timing. While standard fiat payments are deterministic and instant, Proof-of-Work is statistical. The average time to find a share was 2.8 minutes, but the interval was highly erratic (ranging from 12 seconds to nearly 9 minutes). This variance introduces a unique UX friction that must be mitigated by dynamic progress bars.

[3] https://github.com/dappsourcecode/Cryptocurrency-Miners/webapp/mintme/resource-monitoring

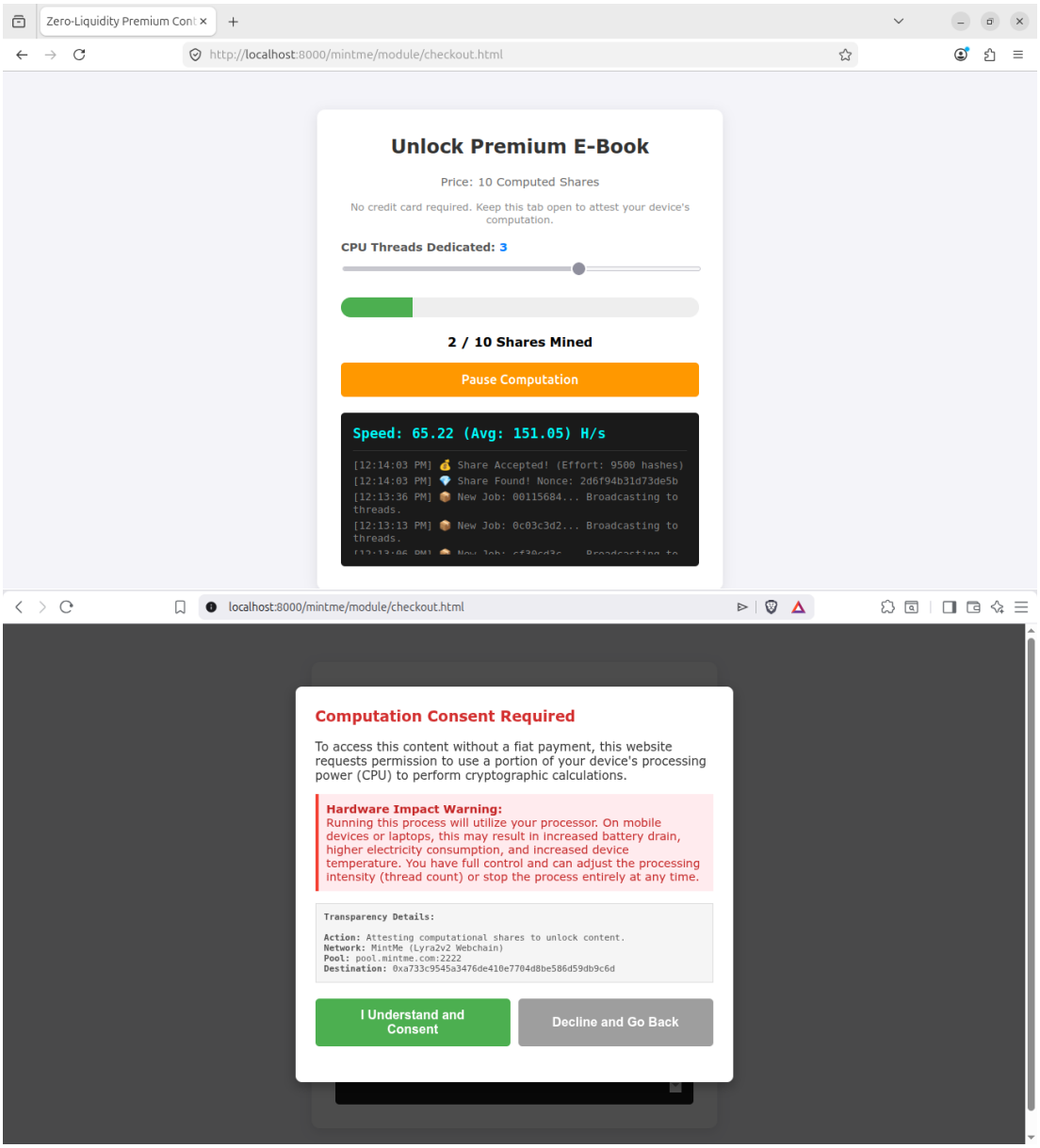


Figure 5: MVP benchmark interface



Figure 6: Resource monitoring screenshot at start of mining

6.3.2 System-Level Resource Consumption

System telemetry captured during the test quantified the physical constraints of web mining. `vmstat` logs revealed that allocating 3 threads drove user-space CPU utilization from an idle 10% to sustained peaks of 77%–100%. `turbostat` data showed the processor exiting its 800 MHz efficiency state, heavily escalating power draw. Conversely, `nethogs` proved the network footprint was practically invisible (under 1 KB/s). This confirms that zero-liquidity mining effectively transfers the transactional cost entirely to the user's local hardware computation and battery reserves, making unthrottled execution on mobile devices highly impractical.

6.4 Economic Feasibility and Benchmarking

To evaluate the true economic feasibility of bridging the "Penny Gap" [18], we mathematically modeled realistic e-business scenarios based on our exact empirical data. As established in Section 5.4, a single MintMe share at a pool difficulty of 12,500 currently yields a fiat equivalent of $0.00000325. At 69.00 H/s, our legacy test hardware required an average of 168.4 seconds per share.

To demonstrate scalability, we compared our empirical legacy hardware (an Intel™ Core i7-4500U from 2013) against a projected modern mid-tier desktop processor, such as the AMD Ryzen™ 5 7600X. A multiplier of 50x (yielding a conservative 3,450 H/s) is applied for the modern CPU. Because cryptographic hashing algorithms like Lyra2v2 are heavily dependent on parallel threading and L3 cache availability (to store the memory-hard state matrix), the architectural jump from a thermally constrained 15-Watt laptop to a 105-Watt actively cooled desktop yields exponential performance gains.

1. **Scenario A: LLM Micro-API Query ($0.0001):** Processing a short chatbot query using a highly optimized Large Language Model (e.g., 200 tokens on GPT-4o-mini) costs

1/100th of a cent. Traditional gateways charge a $0.30 minimum fee, making this impossible to monetize individually. At $0.00000325 per share, this target requires **31 shares**.

- *Legacy CPU (69 H/s):* $\approx$ 1.4 hours.
- *Modern CPU (3,450 H/s):* $\approx$ 1.7 minutes. (Viable for users willing to attest computation prior to executing an AI prompt).

2. **Scenario B: Premium AI Image Generation ($0.01):** Replacing a $20/month AI subscription with a 1-cent zero-liquidity prompt for high-resolution image synthesis. This requires **3,077 shares**.
   - *Legacy CPU (69 H/s):* $\approx$ 143 hours. (Economically unviable).
   - *Modern CPU (3,450 H/s):* $\approx$ 2.8 hours. (Acceptable only for high-intent desktop users willing to leave a background tab open while working).
3. **Scenario C: The Article Paywall ($0.05):** A user mines to bypass a $5/month news subscription paywall [19], requiring **15,384 shares**.
   - *Legacy CPU (69 H/s):* $\approx$ 719 hours.
   - *Modern CPU (3,450 H/s):* $\approx$ 14.3 hours. (Unviable for synchronous reading).

6.4.1 Conclusion on Economic Viability

These rigorous empirical benchmarks yield a crucial conclusion: while the zero-liquidity *architecture* functions flawlessly, its fiat-pegged viability is strictly bound by the current market capitalization of CPU-minable altcoins. At current market rates, mining for standard digital goods (e.g., a $0.25 file) is unviable.

However, this system is exceptionally well-suited for the emerging **Micro-API economy** (Scenario A) and **Bot Mitigation / Proof-of-Work CAPTCHAs**, where the intended transactional value is sub-cent. In these paradigms, where traditional fiat gateways fundamentally fail due to fixed fee structures, zero-liquidity mining provides a highly effective, frictionless alternative to forced subscriptions.

7 Conclusion

The prevailing e-business monetization models—relying heavily on fiat micro-subscriptions and intrusive advertising—are fundamentally constrained by the "Penny Gap" and the Mental Transaction Costs (MTC) of managing financial capital. While traditional Web3 architectures attempted to solve this, they ultimately preserved the prerequisite of financial liquidity.

This manuscript presented an exploratory taxonomy and a practical front-end architecture for zero-liquidity micropayments. By leveraging browser-based cryptocurrency mining via WebAssembly (WASM) and JavaScript, we demonstrated how computation can effectively replace capital. Our taxonomy categorized this mechanism into four distinct e-business pillars: Content Monetization, Philanthropy, Programmatic Utilities, and Digital Security. To transition these concepts from theory to application, we designed and proposed a modular Object-Oriented JavaScript Minimum Viable Product (MVP) that seamlessly bridges the front-end user experience with a backend Stratum proxy.

Crucially, our empirical evaluation grounded these theoretical models in physical reality. System-level telemetry and macroeconomic benchmarking on the live MintMe network revealed

that while zero-liquidity transactions are architecturally sound, they are currently economically infeasible for most legacy consumer hardware and mobile devices when targeting standard fiat-equivalent digital goods (e.g., $0.05 to $0.25). However, the results successfully proved that for modern, high-specification desktop processors, and specifically for sub-cent transactions in the emerging Micro-API economy, computational attestation offers a highly effective, frictionless alternative to forced subscriptions and fiat gateways.

8 Future Work

Despite its vast potential to democratize digital value exchange, the commercial viability of web mining remains hindered by historical cybersecurity stigmas, severe device hardware constraints, and the probabilistic nature of Proof-of-Work (PoW) hash generation.

Most notably, the empirical finding that generating standard fiat-equivalent value is largely infeasible on average consumer hardware provides a strong, definitive imperative for further infrastructural exploration. Because relying solely on networks like MintMe mathematically limits the viable user base, our immediate future work will prioritize evaluating a much broader array of CPU-minable altcoins and alternative decentralized networks. Identifying novel PoW algorithms that are specifically optimized for browser-based WebAssembly environments—aiming for higher computational efficiency and better fiat-conversion rates—will be critical in minimizing thermal and temporal friction for the end user.

Additionally, future research must actively address the cybersecurity stigmas inherited from historical cryptojacking. This involves developing and formalizing standardized "Proof-of-Consent" UI/UX protocols. Establishing open-source guidelines for transparent, real-time telemetry and dynamic thread throttling will allow merchants to safely integrate zero-liquidity mining while ensuring browser vendors and antivirus engines can confidently whitelist legitimate, user-consented applications. Ultimately, optimizing these algorithmic and consent-based frameworks will be essential to fostering global financial inclusion and pioneering new programmatic e-business ecosystems.

9 Acknowledgments

The authors acknowledge the use of Google Gemini as an AI research assistant during the preparation of this manuscript. Specifically, Gemini was utilized to: (1) provide initial structural drafting for several sections, which the authors subsequently reviewed and extensively rewrote; (2) execute an AI-assisted exploratory landscape search to aggregate multivocal literature and identify potential e-business use cases based on the authors' strict criteria; (3) generate foundational TikZ LaTeX code for the UML diagrams, which the authors then modified and finalized; and (4) assist in structuring the vanilla JavaScript class libraries for the MVP prototypes to match the authors' intended architecture. The authors take full responsibility for the final content, originality, and validity of the results presented in this manuscript.

References

[1] Josh Kopelman. The penny gap. First Round Review, 2007. Discussed in the context of e-business pricing elasticity and zero-price effect.

[2] Nick Szabo. Micropayments and mental transaction costs. https://nakamotoinstitute.org/library/micropayments-and-mental-transaction-costs/, 1999. Accessed: 2026-04-24.

[3] Fajar Purnama, Adi Juliawan Pawana, and Adi Panca Saputra Iskandar. Feasibility of cryptocurrency javascript mining for micro payments. In *International Symposium on E-Business Information Systems Evolution (EBISION 2025)*, IFIP Advances in Information and Communication Technology. Springer Nature Switzerland, 2026. In press.

[4] Randall Degges. How to monetize your website with cryptocurrency mining. https://rdegges.com/2017/how-to-monetize-your-website-with-cryptocurrency-mining/, 2017. Accessed: 2026-04-24.

[5] Ananth Vivekanand. Webmine: Browser cryptocurrency mining. https://github.com/AnanthVivekanand/WebMine, 2026. Accessed: 2026-04-24.

[6] VICE. You can now mine cryptocurrency to bail people out of jail. https://www.vice.com/en/article/you-can-now-mine-cryptocurrency-to-bail-people-out-of-jail-bail-bloc/, 2017. Accessed: 2026-04-24.

[7] Quartz. Bail bloc is an easy blockchain app that helps poor people make bail. https://qz.com/1186738/bail-bloc-is-an-easy-blockchain-app-that-helps-poor-people-make-bail, 2018. Accessed: 2026-04-24.

[8] EBISION Deep Research. Exploratory taxonomy and front-end architecture for zero-liquidity micropayments utilizing cryptocurrency web mining. Unpublished Manuscript Data, 2026. Used for mapping UNICEF Hopepage and general taxonomy data.

[9] Scribd. x402: Open standard for ai payments. https://www.scribd.com/document/941536861/x402-whitepaper, 2026. Accessed: 2026-04-24.

[10] MDPI. Combining blockchains, smart contracts, and complex sensors management platform for hyper-connected smartcities: An iot data marketplace use case. *Sensors*, 2021. Accessed: 2026-04-24.

[11] ALTCHA. Proof of work captcha. https://altcha.org/docs/v2/proof-of-work-captcha/, 2026. Accessed: 2026-04-24.

[12] ResearchGate. mcaptcha: Replacing captchas with rate limiters to improve security and accessibility. https://www.researchgate.net/publication/384333437_mCaptcha_Replacing_Captchas_with_Rate_Limiters_to_Improve_Security_and_Accessibility, 2026. Accessed: 2026-04-24.

[13] Friendly Captcha. Friendly captcha - privacy-first bot protection. https://friendlycaptcha.com/, 2026. Accessed: 2026-04-24.

[14] TecharoHQ. Anubis: Weighs the soul of incoming http requests to stop ai crawlers. https://github.com/TecharoHQ/anubis, 2026. Accessed: 2026-04-24.

[15] MintMe.com. Mintme blockchain explorer - network difficulty, 2026. Accessed: 2026-06-04.

[16] MintMe.com. Mintme official mining pool - help and block statistics, 2026. Accessed: 2026-06-04.

[17] MintMe.com. Mintme coin market trading and exchange rates, 2026. Accessed: 2026-06-04.

[18] Stripe. Stripe pricing breakdown: Fees, features, and plans, 2024. Accessed: 2026-06-04.

[19] Adapty. Subscription fatigue: Reasons and how to prevent it, 2025. Accessed: 2026-06-04.